

Cross-Validation: Evaluating Estimator Performance

A structured summary of scikit-learn's User Guide, Section 3.1
(scikit-learn.org/stable/modules/cross_validation.html)

Main Topics & Talking Points

Why cross-validation exists. Evaluating a model on the same data used to train it is a methodological mistake — it overstates performance (overfitting). The fix is to hold out a test set, and, to avoid tuning hyperparameters in a way that leaks test-set information, cross-validation (CV) is used instead of a single fixed validation split.

The basic k-fold idea. Training data is split into k folds; a model is trained on $k-1$ folds and validated on the remaining fold, repeated k times. The reported performance is the average across folds. This uses data more efficiently than a single train/validation/test split.

Computing cross-validated metrics. scikit-learn provides high-level helpers — `cross_val_score`, `cross_validate`, and `cross_val_predict` — that automate fitting and scoring across folds, support custom scoring functions, and can wrap an entire preprocessing + model Pipeline so that transformations (e.g. scaling) are learned only on the training fold.

CV iterators for i.i.d. data. A family of splitter classes (`KFold`, `RepeatedKFold`, `LeaveOneOut`, `LeavePOut`, `ShuffleSplit`) generate the train/test index pairs, assuming samples are independent and identically distributed.

Stratification. For classification tasks with imbalanced classes, `StratifiedKFold` and `StratifiedShuffleSplit` preserve the class proportions in every fold, avoiding folds with missing or rare classes.

Predefined splits. `PredefinedSplit` lets you reuse an existing, user-defined train/validation assignment rather than generating one automatically.

Group-aware cross-validation. When samples are not independent (e.g. multiple samples per patient/subject), `GroupKFold`, `StratifiedGroupKFold`, `LeaveOneGroupOut`, `LeavePGroupsOut`, and `GroupShuffleSplit` ensure that a given group never appears in both the training and test folds, giving a more honest estimate of generalization to unseen groups.

Time series cross-validation. Because time series data is autocorrelated, standard shuffling-based CV leaks future information into training. `TimeSeriesSplit` instead always trains on past folds and tests on a later, non-overlapping fold.

Shuffling considerations. Whether to shuffle before splitting depends on the data: shuffling helps when sample order is arbitrary/grouped by label, but hurts when order carries real temporal information (e.g. news articles by publish date).

CV and model selection. CV iterators are the mechanism underlying hyperparameter search (e.g. Grid Search, covered in the next section of the guide).

Permutation test score. `permutation_test_score` evaluates whether an estimator's cross-validated performance is statistically better than chance by comparing it against scores obtained after randomly shuffling the target labels many times, producing a permutation-based p-value.

Function & Class Reference

Every cross-validation related function, class, and splitter discussed in the article, paired with a one-line summary of what it does.

Core Evaluation Helpers

Name	Description
train_test_split	Randomly splits arrays/data into a single train subset and test subset (a thin wrapper around ShuffleSplit).
cross_val_score	Fits and scores an estimator across CV folds in one call, returning an array of per-fold scores for a single metric.
cross_validate	Like cross_val_score but supports multiple metrics at once and can also return fit/score times, training scores, fitted estimators, and train/test indices.
cross_val_predict	Returns, for every input sample, the prediction made when that sample was in the held-out test fold; useful for visualization or model blending, not for measuring generalization error.
permutation_test_score	Computes a permutation-based p-value showing whether an estimator's CV performance is significantly better than chance, by repeatedly shuffling the target labels.

Cross-Validation Iterators for i.i.d. Data

Name	Description
KFold	Splits all samples into k equal-sized folds; each fold is used once as the test set and the rest as training data.
RepeatedKFold	Runs KFold n_repeats times with different randomized splits each time, to reduce variance in the performance estimate.
RepeatedStratifiedKFold	Runs StratifiedKFold multiple times with different randomization each repetition.
LeaveOneOut (LOO)	Extreme case of KFold where k = n: each test fold is a single sample, so n models are trained on n-1 samples each.
LeavePOut (LPO)	Creates every possible train/test split obtained by removing p samples from the dataset; test sets overlap when p > 1.
ShuffleSplit	Generates a specified number of independent random train/test splits by shuffling then splitting the samples, giving fine control over the number of iterations and split proportions.

Stratified Iterators (Class-Label Aware)

Name	Description
StratifiedKFold	A version of KFold that preserves the percentage of samples of each class in every fold, avoiding folds that lack a class.
StratifiedShuffleSplit	A version of ShuffleSplit that preserves class proportions in each random train/test split.

Predefined Splits

Name	Description
PredefinedSplit	Lets you specify your own fixed train/validation (or CV fold) assignment instead of generating splits automatically, e.g. via a <code>test_fold</code> array.

Group-Aware Iterators

Name	Description
GroupKFold	A KFold variant that guarantees the same group (e.g. subject/patient ID) never appears in both the training and test folds.
StratifiedGroupKFold	Combines StratifiedKFold and GroupKFold: keeps each group in a single fold while trying to preserve class distribution across folds.
LeaveOneGroupOut	Holds out all samples belonging to one group per split, training on the samples from all other groups; equivalent to LeavePGroupsOut with <code>n_groups=1</code> .
LeavePGroupsOut	Holds out samples belonging to <code>p</code> groups for each train/test split, covering all possible combinations of <code>p</code> groups (test sets overlap when <code>p > 1</code>).
GroupShuffleSplit	Combines ShuffleSplit and LeavePGroupsOut: generates randomized train/test partitions where an entire subset of groups is held out for each split.

Time Series Iterator

Name	Description
TimeSeriesSplit	A KFold variant for ordered/time-dependent data: each test fold is the next chunk of data after all prior chunks used for training, so training sets are always supersets of earlier ones and no future data leaks into training.

Related Utilities Mentioned in the Article

Name	Description
Pipeline / make_pipeline	Chains a preprocessing step (e.g. scaling) and an estimator together so that, under cross-validation, the preprocessing is fit only on each training fold and then applied to the corresponding test fold.
StandardScaler	Standardizes features (zero mean, unit variance); shown as an example transformation that must be fit on the training fold only, to avoid leaking test-fold statistics.
make_scorer	Wraps a custom scoring function (e.g. recall_score) into a scorer object usable by cross_validate's scoring parameter.
ClassifierMixin	Base mixin class in scikit-learn; an estimator inheriting from it causes cross_val_score to default to StratifiedKFold instead of plain KFold when cv is given as an integer.

Source: scikit-learn User Guide, Section 3.1 — https://scikit-learn.org/stable/modules/cross_validation.html